Django & Django REST Framework – Complete Theory Notes

Total Questions: 235

Purpose: Brototype revision, technical interviews, and theory assessments.

PART 1 – Django Fundamentals

1. Introduction to Django

Q1. What is Django?

Django is a high-level, open-source Python web framework used to develop secure, scalable, and maintainable web applications quickly.

Q2. Why do we use Django?

- Reduces development time.
- Provides built-in authentication.
- Provides ORM for database operations.
- Includes security features.
- Supports reusable applications.
- Provides an admin interface.

Q3. What are the main features of Django?

1. ORM.
2. Authentication system.
3. Admin interface.
4. URL routing.
5. Template engine.
6. Security features.
7. Middleware.
8. Reusable applications.

Q4. What architecture does Django follow?

Django follows MVT architecture:

- Model – Handles database structure and operations.
- View – Handles application logic.
- Template – Handles presentation and HTML.

Q5. What is the difference between MVC and MVT?

MVC	Django MVT
Model	Model
View	Template
Controller	View

Django itself handles much of the controller functionality.

Q6. What is a Django project?

A Django project is the complete web application containing configuration files, URL configurations, settings, and applications.

Q7. What is a Django application?

A Django application is an individual component inside a project that handles specific functionality.

Examples: `login_app`, `api`, `students`.

Q8. Can one Django project contain multiple applications?

Yes. A Django project can contain multiple reusable applications.

Q9. What is a virtual environment?

A virtual environment is an isolated Python environment that maintains project-specific dependencies without affecting other Python projects.

Q10. Why do we use requirements.txt?

It contains the dependencies and their versions required by a project.

```
None
pip install -r requirements.txt
```

2. Django Project Structure

Q11. What is manage.py?

A command-line utility used to perform Django operations such as running the server, creating applications, migrations, and creating superusers.

Q12. What is settings.py?

It contains project configuration, including database settings, installed applications, middleware, templates, security settings, and static files.

Q13. What is urls.py?

It defines URL patterns and connects incoming URLs to their corresponding views.

Q14. What is wsgi.py?

WSGI stands for Web Server Gateway Interface. It provides an interface between a synchronous Python web application and a compatible web server.

Q15. What is asgi.py?

ASGI stands for Asynchronous Server Gateway Interface. It supports asynchronous applications and protocols such as WebSockets.

Q16. What is the difference between WSGI and ASGI?

WSGI	ASGI
Primarily synchronous	Supports synchronous and asynchronous applications
Traditional HTTP handling	HTTP and asynchronous protocols
Example: Gunicorn	Example: Uvicorn

Q17. What is INSTALLED_APPS?

It is a setting in settings.py that contains the applications enabled in a Django project.

Q18. What is DEBUG?

It controls whether Django displays detailed debugging information.

- True – Development.
- False – Production.

Q19. What is ALLOWED_HOSTS?

It specifies which hostnames or IP addresses are permitted to serve the Django application.

Q20. What is the Django development server?

It is a built-in server used for local development and testing.

```
None
python manage.py runserver
```

It is not recommended for production.

3. Django Request and Response

Q21. What is an HTTP request?

An HTTP request is a message sent by a client to a server to request a resource or perform an operation.

Q22. What is an HTTP response?

An HTTP response is the message returned by the server after processing a request.

Q23. What is a Django view?

A view is a Python function or class that receives an HTTP request, processes application logic, and returns an HTTP response.

Q24. What is a function-based view (FBV)?

A view implemented using a Python function.

Q25. What is a class-based view (CBV)?

A view implemented using a Python class that organizes related request-handling methods.

Q26. Difference between FBV and CBV?

FBV	CBV
Uses functions	Uses classes
Simple to understand	Supports inheritance

Explicit request handling	Uses methods such as get() and post()
Suitable for simple logic	Useful for reusable logic

Q27. What is render()?

render() combines an HTML template with context data and returns an HTTP response.

Q28. What is redirect()?

redirect() sends the browser to another URL or named URL pattern.

Q29. Difference between render() and redirect()?

- render() displays a template.
- redirect() navigates the browser to another URL.

Q30. What is HttpResponse?

It is a Django class used to return an HTTP response to the client.

4. Django Templates

Q31. What is a template in Django?

A template is an HTML file containing dynamic elements that Django processes before sending the final HTML to the browser.

Q32. What is Django Template Language (DTL)?

DTL is Django's built-in template language used to display dynamic data and perform basic presentation logic.

Q33. What are template variables?

They display values passed from a Django view.

Example: `{{ username }}`

Q34. What are template tags?

Template tags control template logic.

Examples: `{% if %}`, `{% for %}`, `{% endif %}`.

Q35. What are template filters?

Filters modify the appearance or representation of template variables.

Examples: `upper`, `lower`, `date`.

Q36. What is template inheritance?

It allows multiple HTML pages to reuse a common parent template.

Q37. What is the purpose of block and extends?

- `{% extends %}` specifies the parent template.
- `{% block %}` defines sections that child templates can override.

Q38. What is context in Django?

Context is a dictionary of data passed from a view to a template.

Q39. What is URL reversing?

Generating a URL using its named URL pattern instead of manually writing the URL.

Q40. What is a URL namespace?

A namespace separates URL names belonging to different applications and prevents naming conflicts.

PART 2 – Django ORM and Database

5. Django Models and ORM

Q41. What is a Django model?

A model is a Python class that defines the structure of a database table.

Q42. What is ORM?

ORM stands for Object Relational Mapper. It allows developers to interact with relational databases using Python objects instead of manually writing SQL queries.

Q43. Why do we use Django ORM?

- Simplifies database operations.
- Reduces manual SQL queries.
- Provides database abstraction.
- Helps manage relationships.
- Improves code readability.

Q44. What is a QuerySet?

A QuerySet is a collection of database objects retrieved through Django ORM.

Q45. What is a model manager?

A manager provides the interface through which database queries are performed. The default manager is `objects`.

Q46. Difference between all() and filter()?

- all() retrieves all records.
- filter() retrieves records matching specified conditions.

Q47. What is get()?

get() retrieves exactly one matching object. It raises an exception if no object or multiple objects are found.

Q48. Difference between get() and filter()?

get()	filter()
Returns one object	Returns a QuerySet
Expects one result	Can return multiple results
Raises exceptions for zero or multiple results	Returns an empty QuerySet when no results exist

Q49. What is create()?

It creates and saves a new database record.

Q50. What is save()?

It saves a model instance into the database.

Q51. What is delete()?

It deletes a model instance or a group of records from the database.

Q52. What is a migration?

A migration is a Django mechanism that applies changes in model definitions to the database schema.

Q53. Difference between makemigrations and migrate?

- makemigrations creates migration files.
- migrate applies migration files to the database.

Q54. What is db.sqlite3?

It is the SQLite database file used when SQLite is configured as the project's database.

Q55. What is a primary key?

A primary key uniquely identifies each record in a database table.

Q56. What is a foreign key?

A foreign key establishes a relationship between two database tables.

Q57. What is str() in Django models?

It defines the human-readable string representation of a model object.

Q58. Difference between null=True and blank=True?

- null=True allows the database field to store NULL.
- blank=True allows the field to be empty during validation.

Q59. What are Django model field types?

Examples include CharField, TextField, IntegerField, EmailField, BooleanField, DateField, DateTimeField, and ForeignKey.

Q60. What is lazy evaluation in QuerySets?

QuerySets are generally not executed when created. Database queries execute when the results are actually needed.

PART 3 – Django Authentication

6. Authentication System

Q61. What is authentication?

Authentication is the process of verifying the identity of a user.

Q62. What is authorization?

Authorization determines what an authenticated user is allowed to access or perform.

Q63. Difference between authentication and authorization?

- Authentication: Who are you?
- Authorization: What are you allowed to do?

Q64. What is Django's built-in User model?

It is a predefined model used to manage user accounts, including username, password, email, names, and permission-related fields.

Q65. What is UserCreationForm?

It is a built-in Django form used to create user accounts. It validates credentials and handles password hashing.

Q66. What is AuthenticationForm?

It is a built-in Django form used to authenticate users using their login credentials.

Q67. What is password hashing?

Password hashing converts a password into a one-way hashed representation before storing it in the database.

Q68. Why should passwords not be stored in plain text?

If the database is compromised, attackers could directly read users' passwords. Hashing helps protect their original values.

Q69. What is authenticate()?

It checks the provided credentials against configured authentication backends and returns a user object if authentication succeeds.

Q70. What is login()?

Django's login() function logs in an authenticated user and associates that user with the current session.

Q71. What is logout()?

It logs out the current user and clears their session data.

Q72. What is a session?

A session stores information about a user's interaction with a web application across multiple HTTP requests.

Q73. Why do we use sessions?

HTTP is stateless. Sessions help Django remember authenticated users between requests.

Q74. What is request.user?

It represents the current user associated with the request. If nobody is logged in, it represents an anonymous user.

Q75. What is is_authenticated?

It is a property that checks whether the current user is authenticated.

Q76. What is login_required?

It is a Django decorator that prevents unauthenticated users from accessing a protected view.

Q77. What is a superuser?

A superuser is an account with full permissions in the Django admin system.

Q78. What is Django Admin?

It is a built-in interface used to manage database records and application data.

Q79. What is an authentication backend?

An authentication backend defines how Django authenticates users and may determine how permissions are checked.

Q80. What is the auth_user table?

It is the default database table where Django stores user account information when using its built-in User model.

PART 4 – Django Forms

7. Forms and Validation

Q81. What is a Django form?

A Django form handles user input, validates submitted data, and helps process that data.

Q82. Why do we use forms in Django?

- Collect user input.
- Validate data.
- Handle form submissions.
- Display form fields.
- Reduce manual validation code.

Q83. What is forms.Form?

It is a base class used to create regular Django forms where fields are explicitly defined.

Q84. What is forms.ModelForm?

It is a form class that automatically creates fields based on a Django model.

Q85. Difference between Form and ModelForm?

Form	ModelForm
Fields manually defined	Fields generated from a model
Does not automatically save to a model	Can save directly to a model
Used for general input	Used for model-related input

Q86. What is form validation?

Form validation checks whether submitted data satisfies the required conditions.

Q87. What is is_valid()?

It validates submitted form data and returns True if valid; otherwise, False.

Q88. What is cleaned_data?

It is a dictionary containing validated and cleaned form data.

Q89. Difference between GET and POST in forms?

- GET sends data through URL query parameters.
- POST sends data in the request body.

POST is generally used for submitting or modifying data.

Q90. What is CSRF?

CSRF stands for Cross-Site Request Forgery. It is an attack where an attacker tricks an authenticated user into submitting an unwanted request.

Q91. What is CSRF protection in Django?

Django provides CSRF middleware and tokens to protect against unauthorized cross-site form submissions.

Q92. What is csrf_token?

`{% csrf_token %}` adds a CSRF token to an HTML form so Django can verify that the request is legitimate.

Q93. What are formsets?

Formsets allow developers to manage multiple instances of the same form together in one request.

Q94. What is a password validator?

A password validator checks whether a password meets specified security requirements, such as minimum length and password similarity checks.

PART 5 – REST API Fundamentals

8. Introduction to REST API

Q95. What is an API?

API stands for Application Programming Interface. It allows two software applications to communicate and exchange information.

Q96. What is a REST API?

A REST API is an API designed around REST architectural principles, generally using HTTP methods to interact with resources.

Q97. What does REST stand for?

Representational State Transfer.

Q98. What are the main characteristics of REST?

1. Client-server architecture.
2. Stateless communication.
3. Cacheability.
4. Uniform interface.
5. Layered system.
6. Optional code on demand.

Q99. What does stateless mean in REST?

Each request should contain the information necessary for the server to process it. The server does not rely on stored client session state to understand the request.

Q100. What is a resource in REST?

A resource is an entity exposed by an API, such as students, products, employees, or orders.

Q101. What is an endpoint?

An endpoint is a specific URL through which a client accesses an API resource.

Example: `/api/students/`

Q102. What is JSON?

JSON stands for JavaScript Object Notation. It is a lightweight data format used for exchanging information between applications.

Q103. Why do REST APIs commonly use JSON?

Because JSON is lightweight, readable, easy to process, and supported by most programming languages.

Q104. Difference between an API and a REST API?

An API is a general communication interface. A REST API is a type of API that follows REST architectural principles.

PART 6 – HTTP Methods and Status Codes

9. HTTP Methods

Q105. What are HTTP methods?

HTTP methods specify the type of operation a client wants to perform on a resource.

Q106. What is GET?

GET retrieves data from the server.

Q107. What is POST?

POST submits data to the server, commonly to create a new resource.

Q108. What is PUT?

PUT replaces an existing resource with the supplied representation.

Q109. What is PATCH?

PATCH partially updates an existing resource.

Q110. What is DELETE?

DELETE removes a resource from the server.

Q111. Difference between PUT and PATCH?

PUT	PATCH
Replaces the resource representation	Partially updates a resource
Usually sends complete data	Usually sends only changed fields

Q112. What is CRUD?

CRUD represents four basic database operations:

CRUD	HTTP method
Create	POST
Read	GET
Update	PUT/PATCH
Delete	DELETE

Q113. What is idempotency?

An operation is idempotent if performing it multiple times produces the same intended result as performing it once.

GET, PUT, and DELETE are generally idempotent. POST is generally not idempotent.

Q114. Difference between safe and idempotent methods?

- Safe methods do not intend to modify server state.
- Idempotent methods produce the same intended result when repeated.

GET is both safe and idempotent.

10. HTTP Status Codes

Q115. What are HTTP status codes?

They are numerical codes returned by a server to indicate the result of an HTTP request.

Q116. What are the main categories of HTTP status codes?

Range	Meaning
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client errors
5xx	Server errors

Q117. What is status code 200?

200 OK – The request was successful.

Q118. What is status code 201?

201 Created – A new resource was successfully created.

Q119. What is status code 204?

204 No Content – The request succeeded, but there is no response body.

Q120. What is status code 400?

400 Bad Request – The request contains invalid data or cannot be processed.

Q121. What is status code 401?

401 Unauthorized – Authentication is required or the supplied credentials are invalid.

Q122. What is status code 403?

403 Forbidden – The server understands the request but refuses to allow the operation.

Q123. What is status code 404?

404 Not Found – The requested resource does not exist.

Q124. What is status code 405?

405 Method Not Allowed – The HTTP method is not supported by that endpoint.

Q125. What is status code 500?

500 Internal Server Error – An unexpected error occurred on the server.

PART 7 – Django REST Framework

11. Introduction to DRF

Q126. What is Django REST Framework?

DRF is a powerful toolkit built on Django for developing Web APIs.

Q127. Why do we use DRF instead of regular Django views?

DRF provides serializers, API-specific request handling, Response objects, authentication, permissions, pagination, filtering, and browsable API support.

Q128. Difference between Django and DRF?

Django	DRF
Web framework	API toolkit built on Django
Commonly renders HTML	Commonly returns JSON
Uses Django forms	Uses serializers for API data
Uses HttpResponse and render()	Provides API Response

Q129. What is APIView?

APIView is a DRF class that allows developers to create API views using HTTP method handlers.

Q130. What is @api_view?

It is a decorator used to convert a function-based view into a DRF API view.

Q131. What is request.data?

It contains parsed request body data sent by the client.

Q132. What is request.method?

It identifies the HTTP method used in the incoming request.

Q133. What is DRF's Response?

Response is a DRF class that returns serialized data or other content to the client.

Q134. Difference between HttpResponse and DRF Response?

- HttpResponse is Django's standard HTTP response class.
- DRF Response supports content negotiation and renders data using the selected renderer.

Q135. What is status in DRF?

It provides named constants for HTTP status codes.

Examples: `HTTP_200_OK`, `HTTP_201_CREATED`, `HTTP_404_NOT_FOUND`.

PART 8 – Serializers

12. Django REST Framework Serializers

Q136. What is a serializer?

A serializer converts complex data types, such as Django model instances and QuerySets, into native Python data types that can be rendered into formats such as JSON.

It can also deserialize incoming data and validate it.

Q137. Why do we use serializers?

- Convert model objects into JSON-compatible data.
- Validate incoming API data.
- Convert validated data into Python objects.
- Create and update model instances.

Q138. What is ModelSerializer?

ModelSerializer is a DRF serializer that automatically generates fields based on a Django model.

Q139. Difference between Serializer and ModelSerializer?

Serializer	ModelSerializer
Fields manually defined	Fields generated from models
More manual configuration	Less repetitive code
General-purpose serializer	Designed for model-related operations

Q140. What is the Meta class in a ModelSerializer?

It provides configuration for the serializer, such as the model and fields to include.

Q141. What does fields = 'all' mean?

It includes all model fields in the serializer.

Q142. What is serializer.is_valid()?

It validates incoming data against serializer field definitions and validation rules.

Q143. What is serializer.validated_data?

It contains data that successfully passed serializer validation.

Q144. What is serializer.errors?

It contains validation error details when submitted data is invalid.

Q145. What does serializer.save() do?

It saves validated data. Depending on whether an existing instance was supplied, it calls create() or update().

Q146. What is serializer.data?

It provides the serialized representation of the object or validated data.

Q147. Difference between serializer.data and serializer.validated_data?

- validated_data contains validated Python data used for saving or processing.
- data contains the serialized representation suitable for an API response.

Q148. What is many=True?

It tells a serializer to handle multiple objects instead of a single object.

Q149. What is raise_exception=True?

It raises a DRF validation exception when submitted data is invalid. DRF handles the exception and generally returns HTTP 400.

Q150. What is serializer validation?

It checks incoming data before it is saved to the database.

PART 9 – CRUD API Development

13. Function-Based CRUD Views

Q151. What is a CRUD API?

A CRUD API provides endpoints to create, retrieve, update, and delete resources.

Q152. What happens in a GET request?

The API retrieves information from the database and returns it to the client.

Q153. What happens in a POST request?

The API receives data, validates it using a serializer, saves it, and returns the created resource.

Q154. What happens in a PUT request?

The API receives complete updated data, validates it, and replaces the existing resource representation.

Q155. What happens in a PATCH request?

The API receives partial data and updates only the supplied fields.

Q156. What happens in a DELETE request?

The API identifies the requested object and deletes it from the database.

Q157. What is get_object_or_404()?

It retrieves an object matching given conditions. If it does not exist, it raises an HTTP 404 exception.

Q158. Why do we use partial=True in serializers?

It allows partial updates without requiring all required fields.

Q159. What is API validation?

It ensures incoming data meets required conditions before database operations are performed.

Q160. Why should we handle exceptions in API views?

Exception handling prevents unexpected errors from breaking the application and allows meaningful error responses.

PART 10 – Authentication and Permissions in DRF

14. API Security

Q161. What is API authentication?

API authentication verifies the identity of the client making an API request.

Q162. What is API authorization?

API authorization determines whether an authenticated client has permission to perform a particular operation.

Q163. Difference between Django authentication and DRF authentication?

Django authentication primarily manages users and sessions for Django applications.

DRF authentication identifies clients making API requests using supported mechanisms such as session authentication, token authentication, or custom authentication.

Q164. What is API key authentication?

API key authentication uses a unique key sent with an API request to identify or authorize a client.

Q165. Difference between API key and username/password authentication?

- Username/password authentication uses user credentials.
- API key authentication uses a generated key associated with a client or user.

An API key is not automatically equivalent to a full user login.

Q166. What is a permission class in DRF?

A permission class determines whether a request should be allowed to access a particular API view.

Q167. What is AllowAny?

It allows any client to access the API, whether authenticated or not.

Q168. What is IsAuthenticated?

It allows access only to authenticated users.

Q169. What is IsAdminUser?

It allows access to users who have staff status.

Q170. Difference between authentication and permissions in DRF?

Authentication identifies who is making the request.

Permissions decide whether that request is allowed.

Q171. What is the purpose of permission_classes?

It specifies the permission rules applied to an API view.

Q172. Why is API security important?

It protects sensitive information, prevents unauthorized access, and controls which operations clients can perform.

PART 11 – ViewSets and Routers

15. Advanced DRF Concepts

Q173. What is a ViewSet?

A ViewSet is a DRF class that groups related API actions into one class.

Q174. What is ModelViewSet?

It is a DRF ViewSet that automatically provides standard CRUD operations for a model when configured with a queryset and serializer.

Q175. What is a router in DRF?

A router automatically generates URL patterns for registered ViewSets.

Q176. Difference between APIView and ViewSet?

APIView	ViewSet

Uses HTTP method handlers	Uses action methods
URLs generally configured manually	Routers can generate URLs
More control over individual methods	Reduces repetitive CRUD code

Q177. What is pagination?

Pagination divides a large collection of API results into smaller pages.

Q178. Why is pagination important?

- Reduces response size.
- Improves performance.
- Reduces unnecessary data transfer.
- Makes large datasets easier to navigate.

Q179. What is filtering in DRF?

Filtering allows clients to retrieve only records matching specific conditions.

Q180. Difference between filtering and pagination?

Filtering selects which records should be returned.

Pagination divides the selected records into smaller groups.

PART 12 – Testing APIs

16. Postman and API Testing

Q181. What is Postman?

Postman is a tool used to send HTTP requests, test APIs, inspect responses, and debug API behaviour.

Q182. Why do we use Postman?

- Test GET requests.
- Send POST requests.
- Test PUT and PATCH requests.

- Test DELETE requests.
- Add headers and authentication.
- Inspect response status codes.
- Verify JSON responses.

Q183. What is a request body?

A request body contains the data sent by the client to the server.

Q184. What are HTTP headers?

Headers provide additional information about an HTTP request or response.

Examples: Content-Type, Accept, Authorization.

Q185. What is Content-Type: application/json?

It tells the server that the request body contains JSON data.

Q186. Difference between request headers and request body?

Headers provide metadata about the request.

The body contains the actual data being submitted.

Q187. What is API testing?

API testing checks whether API endpoints work correctly according to their expected behaviour.

Q188. What should we test in a CRUD API?

- Successful creation.
- Successful retrieval.
- Successful updating.
- Successful deletion.
- Invalid input.
- Missing records.
- Unauthorized requests.
- Unsupported HTTP methods.

PART 13 – API Documentation and Swagger

17. API Documentation

Q189. What is API documentation?

API documentation is a detailed explanation of an API's endpoints, HTTP methods, request parameters, authentication requirements, responses, and error codes.

Q190. Why is API documentation important?

- Helps developers understand the API.
- Explains how to use endpoints.
- Reduces communication between developers.
- Makes API integration easier.
- Helps with API testing.

Q191. What is Swagger?

Swagger is a collection of tools used to describe, document, and interact with APIs.

Q192. What is OpenAPI?

OpenAPI is a specification used to describe REST APIs in a standardized format.

Q193. Difference between Swagger and OpenAPI?

OpenAPI is the API description specification.

Swagger refers to tools that work with OpenAPI specifications, such as Swagger UI.

Q194. What is Swagger UI?

Swagger UI provides an interactive web interface where developers can explore API endpoints and send requests.

Q195. What is drf-yasg?

It is a third-party Django REST Framework package that generates Swagger and ReDoc API documentation.

Q196. What is the purpose of swagger_auto_schema?

It allows developers to customize API documentation for individual views, including descriptions, request parameters, and response information.

Q197. What is ReDoc?

ReDoc is a tool that generates readable API documentation from an OpenAPI specification.

Q198. Can we test APIs through Swagger UI?

Yes. Swagger UI provides a Try it out feature that allows developers to send requests and inspect responses.

PART 14 – Django Deployment

18. AWS and Production Deployment

Q199. What is deployment?

Deployment is the process of making an application available in a production environment so users can access it.

Q200. What is AWS?

AWS stands for Amazon Web Services. It is a cloud computing platform that provides virtual servers, databases, storage, networking, and security services.

Q201. What is Amazon EC2?

EC2 stands for Elastic Compute Cloud. It provides virtual servers that can be used to host applications.

Q202. What is an EC2 instance?

An EC2 instance is a virtual machine running in AWS.

Q203. What is SSH?

SSH stands for Secure Shell. It is a protocol used to securely connect to and manage remote servers.

Q204. What is a .pem file?

A PEM file can contain a private key used for SSH authentication when connecting to an EC2 instance.

Q205. What is a security group?

A security group is a virtual firewall that controls inbound and outbound traffic for AWS resources such as EC2 instances.

Q206. What is a port?

A port is a communication endpoint used by applications and services on a computer.

Port	Purpose
22	SSH
80	HTTP
443	HTTPS
8000	Common Django development server port

Q207. What is Gunicorn?

Gunicorn is a Python WSGI HTTP server used to run Python web applications in production.

Q208. Why do we use Gunicorn instead of Django's development server?

Gunicorn is designed to serve production applications and manage application worker processes. Django's built-in server is intended for development.

Q209. What is Nginx?

Nginx is a web server that can act as a reverse proxy in front of an application server.

Q210. What is a reverse proxy?

A reverse proxy receives client requests and forwards them to the appropriate backend server.

Q211. Difference between Nginx and Gunicorn?

- Nginx handles incoming web traffic and can serve static files.
- Gunicorn runs the Python Django application.

Q212. What is systemd?

Systemd is a Linux service manager used to manage background services.

Q213. Why did we create a systemd service for Django?

To run Django through Gunicorn as a managed background service and automatically restart it when configured to do so.

Q214. What is an environment variable?

An environment variable is a configuration value stored outside the application's source code.

Q215. Why should we store SECRET_KEY in an environment variable?

It prevents the secret key from being hardcoded into the source code and accidentally exposed.

Q216. Why should DEBUG=False be used in production?

It prevents Django from displaying detailed debugging information that could expose sensitive application details.

Q217. What is collectstatic?

It collects static files from Django applications into the directory configured by STATIC_ROOT.

```
None
python manage.py collectstatic
```

Q218. What are static files?

Static files are files such as CSS, JavaScript, and images used by the application.

Q219. What are media files?

Media files are user-uploaded files such as profile pictures and documents.

Q220. What is HTTPS?

HTTPS is the secure version of HTTP. It uses TLS encryption to protect communication between clients and servers.

Q221. What is an SSL/TLS certificate?

It is a digital certificate used to establish a secure connection and verify a website's identity.

Q222. What is Elastic Beanstalk?

AWS Elastic Beanstalk is a managed service that helps deploy and manage applications without manually managing every infrastructure component.

Q223. Difference between EC2 and Elastic Beanstalk?

EC2	Elastic Beanstalk

Virtual server	Managed application deployment service
More manual configuration	Automates many deployment tasks
Greater infrastructure control	Less infrastructure management

Q224. What is a production environment?

A production environment is the live environment where real users access the application.

Q225. Why should we use ALLOWED_HOSTS correctly?

It protects the Django application against requests containing unauthorized host headers.

PART 15 – Important Differences for Exams and Interviews

Concepts	Main difference
Django vs DRF	Web framework vs API toolkit
Project vs App	Entire application vs individual component
MVT vs MVC	Different separation of responsibilities
WSGI vs ASGI	Synchronous interface vs async-capable interface
FBV vs CBV	Function vs class
Render vs Redirect	Return template vs navigate to another URL
GET vs POST	Retrieve vs submit
PUT vs PATCH	Full replacement vs partial update
Authentication vs Authorization	Identity verification vs access control
Form vs ModelForm	Regular form vs model-connected form
Serializer vs ModelSerializer	Manually defined vs model-generated fields
HttpResponse vs Response	Django response vs DRF response

get() vs filter()	Single object vs QuerySet
makemigrations vs migrate	Create migration files vs apply migrations
API key vs Session authentication	Key-based authentication vs session-based authentication
APIView vs ViewSet	Explicit method handling vs grouped API actions
Swagger vs OpenAPI	Documentation tools vs specification
Gunicorn vs Nginx	Application server vs web server/reverse proxy
EC2 vs Elastic Beanstalk	Virtual machine vs managed deployment service

---

PART 16 – Practical Understanding Questions

Q226. Why do we use serializers instead of directly returning model objects?

Model objects are complex Python objects that cannot be directly rendered as JSON. Serializers convert them into JSON-compatible data.

Q227. Why do we check serializer.is_valid() before saving?

To ensure that incoming data satisfies validation requirements before modifying the database.

Q228. Why do we use many=True when retrieving multiple students?

Because the serializer must know that it is receiving multiple objects rather than one object.

Q229. Why do we use get_object_or_404() when retrieving an individual student?

To retrieve the student if it exists and automatically return a 404 response if it does not.

Q230. Why do we use partial=True for PATCH?

Because PATCH only updates selected fields, so the remaining required fields should not be mandatory.

Q231. Why should we separate URLs, views, serializers, and models?

It follows separation of concerns, making the application easier to understand, maintain, test, and extend.

Q232. Why do we use environment variables for production settings?

To separate sensitive configuration from source code and make deployment configuration easier to manage.

Q233. Why do we need both Nginx and Gunicorn?

Nginx handles external HTTP traffic and forwards application requests to Gunicorn, while Gunicorn executes the Django application.

Q234. Why do we use Postman even when we have Swagger?

Postman provides a flexible environment for testing, organizing, and debugging API requests. Swagger is useful for interactive API documentation and testing.

Q235. What happens when a client sends a POST request to a Django REST API?

1. The request reaches the API endpoint.
2. DRF processes the request.
3. The view receives the request data.
4. The serializer validates the data.
5. Valid data is saved to the database.
6. The API returns a response with an appropriate status code.

Final Revision Checklist

- Django fundamentals and MVT architecture
- Project structure and configuration
- Request-response lifecycle
- Templates and template inheritance
- Models and ORM
- Migrations and database relationships
- Authentication and sessions
- Forms and validation
- CSRF protection
- REST API fundamentals
- HTTP methods and status codes
- DRF and APIView
- Serializers and ModelSerializers

- CRUD API development
- Authentication and permissions
- ViewSets and routers
- Pagination and filtering
- Postman and API testing
- Swagger and OpenAPI
- AWS EC2 deployment
- Gunicorn, Nginx and systemd
- Environment variables and production security

End of Django & DRF Theory Notes – 235 Questions.